

INFORME TÉCNICO DE BLUE TEAM

Laboratorio Semestral AWSGoat — Módulos 1 y 2 (Blog Serverless y Aplicación HR)

Asignatura: Administración y Entornos de Seguridad en la Nube

Profesor: José Moreno

Integrante(s): Cleidys Guzmán, Monica Pineda, Victor Pineda

Fecha: 16 de julio de 2026

Rol desempeñado: Blue Team — Análisis, monitoreo, remediación y fortalecimiento de infraestructura AWS

Contenido

Introducción	3
PARTE I – MÓDULO 1: BLOG SERVERLESS	4
1. Arquitectura desplegada.....	4
2. Vulnerabilidades identificadas	4
3. Controles y remediaciones aplicadas	5
3.1 XSS reflejado en la barra de búsqueda (SearchNotFound.js)	5
3.2 Inyección SQL (PartiQL) en /search-author	6
3.3 IDOR en /change-password	8
3.4 Exposición de datos sensibles en /dump	10
3.5 Hallazgo adicional: deficiencias de Terraform que impedían la propagación de los parches	11
PARTE II – MÓDULO 2: APLICACIÓN HR	13
4. Arquitectura desplegada.....	13
5. Vulnerabilidades identificadas	13
6. Controles y remediaciones aplicadas	14
6.1 Reducción de privilegios en el rol de instancia ECS	15
6.2 Restricción de la permissions boundary del rol de instancia.....	16
6.3 Eliminación de permisos administrativos totales en el rol ec2-deployer	17
6.4 Restricción del rol de tarea ECS sobre Secrets Manager	18
6.5 Cifrado, bloqueo de acceso público, versionado y registro de acceso en el bucket de estado de Terraform	19
6.6 Bucket dedicado de logs y protección de acceso público	20
6.7 Habilitación de registro de acceso del Application Load Balancer	21
6.8 Implementación de CloudTrail con entrega a S3 y CloudWatch Logs	22
6.9 Detección de cambios de IAM mediante Metric Filter y alarma	23
6.10 Monitoreo de disponibilidad del servicio (ALB / ECS)	25
6.11 Registro de logs del contenedor de aplicación (ECS)	26
6.12 Notificación centralizada mediante Amazon SNS	27
6.13 Tagging y nomenclatura estándar	28
Hallazgos Módulo 1	30
Hallazgos Módulo 2	31
Recomendaciones Módulo 1	31
Recomendaciones Módulo 2	32
Conclusiones Módulo 1	32
Conclusiones Módulo 2	33

Introducción

AWSGoat es un entorno deliberadamente vulnerable, desarrollado por INE Labs, orientado al aprendizaje práctico de pruebas de penetración y auditoría de seguridad en infraestructuras de Amazon Web Services (AWS). El proyecto está compuesto por dos módulos independientes desplegados mediante Terraform: el Módulo 1, que simula una aplicación de blog basada en servicios serverless (Lambda, API Gateway, DynamoDB y S3), y el Módulo 2, que simula una aplicación de recursos humanos (HR) desplegada sobre contenedores gestionados con Amazon ECS, balanceador de carga y base de datos relacional (RDS). Ambos módulos incorporan de forma intencional configuraciones y desarrollos inseguros alineados con las categorías del OWASP Top 10:2021, con el propósito de que el estudiante identifique, explote y posteriormente corrija dichas debilidades.

El presente informe documenta el trabajo realizado como integrante del Blue Team sobre ambos módulos del laboratorio AWSGoat. En el Módulo 1, el trabajo correspondió a los cuatro primeros retos asignados: Cross-Site Scripting (XSS) reflejado, Inyección SQL (SQL Injection), Insecure Direct Object Reference (IDOR) y Exposición de Datos Sensibles. El objetivo fue identificar las vulnerabilidades de código explotadas por el Red Team, aplicar los parches correspondientes en el código fuente (React) y en el backend (Lambda/Python), y validar que los ataques originales dejaran de ser explotables tras el despliegue de los cambios mediante Terraform. En el Módulo 2, el objetivo fue identificar las configuraciones inseguras presentes en la infraestructura desplegada (aplicación HR), aplicar controles de remediación sobre el código de infraestructura como código (Terraform) y validar, mediante consola de AWS y AWS CLI, que dichos controles operaran correctamente.

Adicionalmente, durante el proceso de remediación del Módulo 1 se identificó un conjunto de deficiencias en el propio pipeline de Infraestructura como Código (IaC) del laboratorio, que impedían que los parches de código aplicados se reflejaran en el entorno desplegado sin intervención manual. Este hallazgo, ajeno a las vulnerabilidades originalmente asignadas, se documenta en la sección 4.5 por su relevancia para la integridad del proceso de remediación.

El alcance del trabajo del Blue Team comprendió, para el Módulo 1: (i) la revisión y modificación del código fuente afectado (SearchNotFound.js y lambda_function.py); (ii) el re-empaquetado y re-despliegue de dichos componentes mediante Terraform; (iii) la corrección de las deficiencias del pipeline de IaC que impedían la propagación de los cambios; y (iv) la verificación funcional de cada remediación, repitiendo el ataque original documentado por el Red Team y confirmando que ya no resultaba explotable. Para el Módulo 2, el alcance comprendió: (i) la revisión del archivo main.tf original; (ii) la identificación de configuraciones de IAM, S3, ALB, CloudTrail y CloudWatch que representaban riesgos de seguridad; (iii) la modificación del código Terraform para reducir privilegios excesivos, habilitar cifrado, registro y monitoreo; y (iv) la verificación funcional de las remediaciones mediante consola de AWS y AWS CLI, incluyendo dos pruebas dirigidas de generación de alarmas (creación de un usuario IAM de prueba y detención forzada del servicio ECS). Las vulnerabilidades explotadas por el Red Team durante la fase de ataque en el Módulo 2 se mencionan únicamente como contexto necesario para explicar su detección y remediación, sin detallar el procedimiento de explotación.

Cada control aplicado se acompaña, cuando existe evidencia, de las capturas de pantalla correspondientes a su verificación, tanto por consola (GUI) como por AWS CLI, de modo que el lector pueda contrastar directamente lo descrito con lo efectivamente ejecutado.

PARTE I – MÓDULO 1: BLOG SERVERLESS

Informe Técnico de Blue Team — Componentes serverless (Lambda, API Gateway, DynamoDB, S3)

1. Arquitectura desplegada

La infraestructura del Módulo 1 fue desplegada mediante Terraform sobre una única cuenta de AWS, región us-east-1. Los servicios efectivamente desplegados e involucrados en el ejercicio de remediación son los siguientes:

- Frontend: aplicación React empaquetada como build estático, alojada en un bucket de Amazon S3 (production-blog-awsgoat-bucket-<cuenta>) y servida a través de una función Lambda (blog-application, Node.js) integrada con API Gateway.
- Backend de datos: función Lambda blog-application-data (Python 3.9), que expone los endpoints de la aplicación (login, search-author, change-password, dump, entre otros) a través de Amazon API Gateway.
- Persistencia: tablas de Amazon DynamoDB (blog-users, blog-posts), consultadas desde la Lambda de datos mediante PartiQL (ExecuteStatement).
- Autenticación: JSON Web Tokens (JWT) firmados con una clave simétrica (JWT_SECRET) almacenada como variable de entorno de la función Lambda.
- Cómputo auxiliar: una instancia EC2 (goat_instance) utilizada como parte de otros retos del módulo, no cubierta por el alcance de este informe.
- Empaquetado: un recurso data "archive_file" por cada función Lambda, que comprime el código fuente en un .zip subido como filename del recurso aws_lambda_function correspondiente.

Este informe cubre exclusivamente los componentes de frontend (SearchNotFound.js) y de la Lambda de datos (lambda_function.py) relacionados con los cuatro retos remediados; no se documentan controles sobre EC2, IAM ni servicios de auditoría, que corresponden a otros módulos o retos del laboratorio.

2. Vulnerabilidades identificadas

A partir de la guía de remediación asignada y de la explotación previa documentada por el Red Team, se identificaron las siguientes cuatro vulnerabilidades de código en el Módulo 1. Estas se describen desde la perspectiva de detección y remediación del Blue Team; el procedimiento de explotación se referencia únicamente como contexto necesario.

Vulnerabilidad	Servicio	Riesgo asociado	OWASP	Buena práctica AWS relacionada
XSS reflejado en barra de búsqueda	Frontend (S3 / React)	Medio. Permite ejecutar JavaScript arbitrario en el navegador de otro usuario a partir de un input no	A03:2021 – Injection	Escapado automático del framework de UI; nunca insertar HTML crudo desde input de usuario

		sanitizado, habilitando robo de sesión o phishing.		
Inyección SQL (PartiQL) en /search-author	Lambda (blog-application-data) / DynamoDB	Alto. Concatenación directa de input de usuario en una sentencia PartiQL permite alterar la lógica de la consulta y extraer datos no autorizados.	A03:2021 – Injection	Uso de consultas parametrizadas (prepared statements); nunca concatenar input en sentencias de consulta
IDOR en /change-password	Lambda (blog-application-data)	Alto. El identificador del usuario a modificar se tomaba del cuerpo de la petición, permitiendo a cualquier usuario autenticado cambiar la contraseña de otro.	A01:2021 – Broken Access Control	Derivar la identidad del sujeto de la operación del token de sesión firmado, nunca de un campo editable por el cliente
Exposición de datos sensibles en /dump	Lambda (blog-application-data) / DynamoDB	Crítico. El endpoint retornaba el volcado completo de la tabla de usuarios sin exigir autenticación alguna.	A01:2021 – Broken Access Control / A02:2021 – Cryptographic Failures	Autenticación y autorización obligatorias en todo endpoint que exponga datos de usuarios

3. Controles y remediaciones aplicadas

Para cada vulnerabilidad se modificó el código fuente correspondiente (React o Python) y se re-desplegó mediante Terraform. A continuación se presentan los extractos relevantes en su versión original y corregida, la explicación del problema, la acción realizada y el resultado de la verificación.

3.1 XSS reflejado en la barra de búsqueda (SearchNotFound.js)

Problema identificado

El componente SearchNotFound.js renderizaba el término de búsqueda ingresado por el usuario directamente como HTML crudo mediante la propiedad dangerouslySetInnerHTML de React, sin ningún tipo de sanitización o escapeado.

Antes

```
<p style={{ textAlign: 'center' }}>Results for <strong dangerouslySetInnerHTML={{ __html:
searchQuery }}/></p>
```

Después

```
<p style={{ textAlign: 'center' }}>Results for <strong>{searchQuery}</strong></p>
```

```
export default function SearchNotFound({ searchQuery = '', ...other }) {
  return (
    <Paper { ...other}>
      {// eslint-disable-next-line}
      <p style={{ textAlign: 'center' }}>Results for <strong>{searchQuery}</strong></p>
    </Paper>
  )
}
```

Figura 1: edición del archivo `src/src/components/SearchNotFound.js`, línea 15

Al reemplazar `dangerouslySetInnerHTML` por la interpolación estándar de JSX (`{searchQuery}`), React aplica escapado automático de caracteres especiales (<, >, comillas), de modo que cualquier payload ingresado se renderiza como texto plano y no como código HTML/JavaScript ejecutable.

Verificación

Se repitió el ataque original ingresando el payload `<img src='a' onerror=alert('xss')>` en la barra de búsqueda, sobre el entorno redesplegado. Tras el fix, el payload se muestra como texto literal en pantalla y no se ejecuta ningún script, confirmando la remediación.



Figura 2: Ingreso del payload y rechazo en la barra de búsqueda

Nota adicional: se identificaron otros dos usos de `dangerouslySetInnerHTML` en el proyecto (`blogpost.js` y `HomePage.js`, para el contenido de los posts) que representan un riesgo de Stored XSS. No formaban parte del alcance de los cuatro retos asignados, pero se documentan aquí como hallazgo adicional para consideración futura.

3.2 Inyección SQL (PartiQL) en `/search-author`

Problema identificado

El parámetro `name`, recibido del cliente, se concatenaba directamente dentro de la sentencia PartiQL ejecutada contra DynamoDB, permitiendo alterar la lógica de la consulta mediante un payload como `'1'='1`.

Antes

```

try:
    if authLevel == "200":
        exec_statement = (
            'SELECT * FROM "blog-users" where name = \''
            + name
            + "' and authLevel in ('200','100');"
        )
    elif authLevel == "100":
        exec_statement = (
            'SELECT * FROM "blog-users" where name = \''
            + name
            + "' and authLevel in ('200','100','0');"
        )
    else:
        exec_statement = (
            'SELECT * FROM "blog-users" where name = \'' + name + "';"
        )

    responses = client.execute_statement(Statement=exec_statement)

```

Después

```

try:
    if authLevel == "200":
        exec_statement = 'SELECT * FROM "blog-users" where name = ? and
authLevel in (\'200\',\'100\');'
    elif authLevel == "100":
        exec_statement = 'SELECT * FROM "blog-users" where name = ? and
authLevel in (\'200\',\'100\',\'0\');'
    else:
        exec_statement = 'SELECT * FROM "blog-users" where name = ?;'

    responses = client.execute_statement(
        Statement=exec_statement,
        Parameters=[{"S": name}]
    )

```

```

if "value" not in data or "authLevel" not in data:
    responses = "Something is missing. Please check proper authentication"
    return generateResponse(200, json.dumps({"body": responses}))
name = data["value"]
authLevel = data["authLevel"]
try:
    if authLevel == "200":
        exec_statement = 'SELECT * FROM "blog-users" where name = ? and authLevel in (\'200\',
\'100\');'
    elif authLevel == "100":
        exec_statement = 'SELECT * FROM "blog-users" where name = ? and authLevel in (\'200\',
\'100\',\'0\');'
    else:
        exec_statement = 'SELECT * FROM "blog-users" where name = ?;'

    responses = client.execute_statement(
        Statement=exec_statement,
        Parameters=[{"S": name}]
    )

```

Figura 3: edición del archivo `src/src/components/SearchNotFound.js`, línea 503

Se sustituyó la concatenación de cadenas por una consulta parametrizada, utilizando el marcador ? y el argumento Parameters del cliente de ejecución de PartiQL. Con este cambio, el valor de name viaja como dato tipado y no como parte del texto de la sentencia, de modo que cualquier carácter de control SQL se interpreta como texto literal de búsqueda.

Verificación

Se interceptó una petición a /search-author con Burp Suite, y se sustituyó el valor de name por el payload hello' or '1'='1. Tras el fix, la respuesta ya no devuelve el conjunto completo de usuarios; el payload se trata como cadena de búsqueda literal, sin coincidencias.

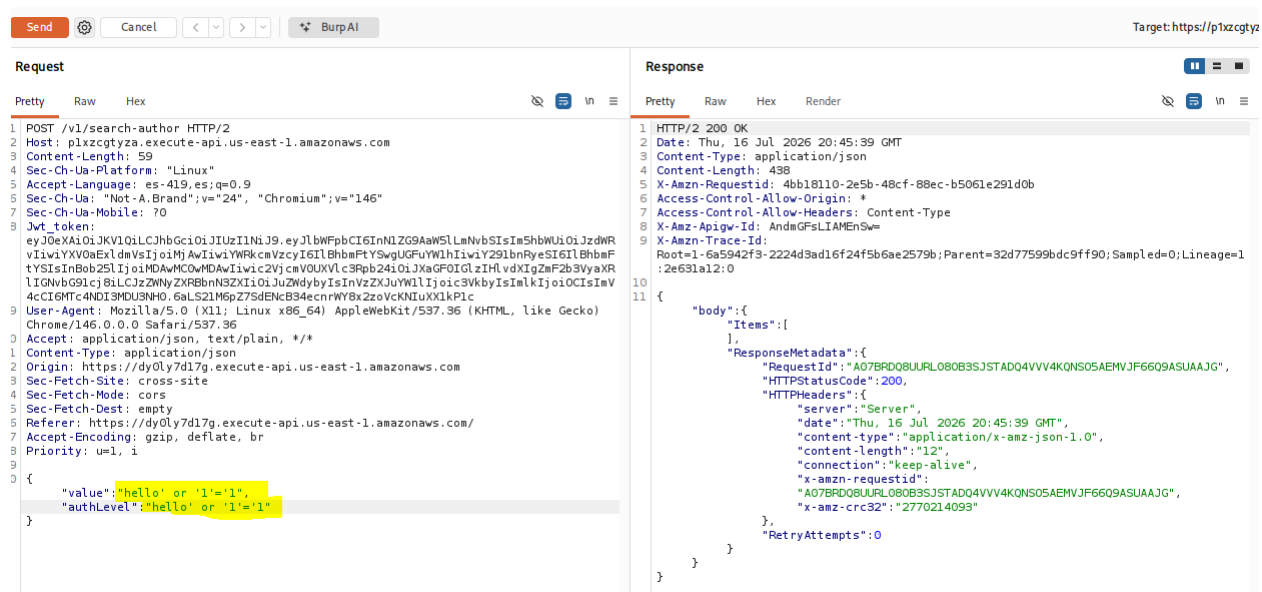


Figura 4: Payload fallido (Burp Repeater)

3.3 IDOR en /change-password

Problema identificado

El identificador (id) del usuario cuya contraseña se modificaba se tomaba directamente del cuerpo JSON enviado por el cliente, sin validar que correspondiera al usuario autenticado. Cualquier usuario con sesión iniciada podía interceptar la petición y sustituir el id por el de otro usuario para cambiar su contraseña.

Antes

```
if event["httpMethod"] == "POST" and
    event["path"] == "/change-password":
    data = json.loads(event["body"])

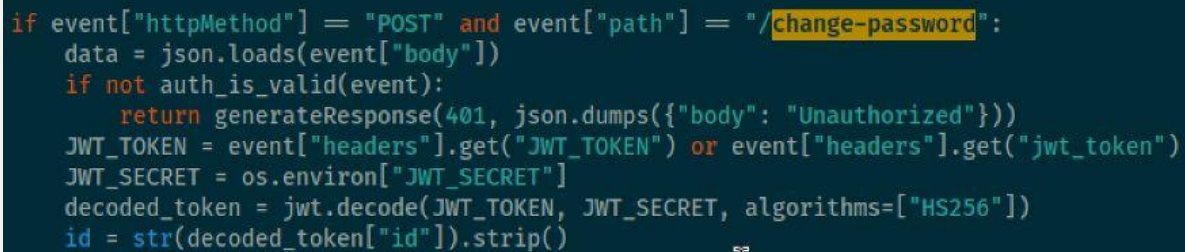
    id = data["id"].strip()
```


Después

```
if event["httpMethod"] == "POST" and
    event["path"] == "/change-password":
    data = json.loads(event["body"])

    if not auth_is_valid(event):
        return generateResponse(401,
                                json.dumps({"body": "Unauthorized"}))

    JWT_TOKEN = event["headers"].get("JWT_TOKEN") \
        or event["headers"].get("jwt_token")
    JWT_SECRET = os.environ["JWT_SECRET"]
    decoded_token = jwt.decode(JWT_TOKEN, JWT_SECRET,
                               algorithms=["HS256"])
    id = str(decoded_token["id"]).strip()
```



```
if event["httpMethod"] == "POST" and event["path"] == "/change-password":
    data = json.loads(event["body"])
    if not auth_is_valid(event):
        return generateResponse(401, json.dumps({"body": "Unauthorized"}))
    JWT_TOKEN = event["headers"].get("JWT_TOKEN") or event["headers"].get("jwt_token")
    JWT_SECRET = os.environ["JWT_SECRET"]
    decoded_token = jwt.decode(JWT_TOKEN, JWT_SECRET, algorithms=["HS256"])
    id = str(decoded_token["id"]).strip()
```

Figura 5: resources/lambda/data/lambda_function.py, línea ~329

Se añadió una validación previa del token de sesión mediante la función `auth_is_valid` (que verifica firma y expiración del JWT), y se sustituyó la fuente del `id`: en lugar de tomarse del cuerpo de la petición, se extrae del token JWT firmado del usuario autenticado. De esta forma, el identificador de la cuenta a modificar deja de ser un valor controlable por el cliente.

Verificación

Se interceptó la petición de cambio de contraseña propia con Burp Suite y se sustituyó el campo `id` del cuerpo por el de otro usuario de prueba. Tras el fix, la petición retorna 401 Unauthorized cuando no se envía un token válido correspondiente al usuario objetivo, y el cambio de contraseña solo afecta a la cuenta asociada al JWT enviado, no al valor del campo `id` del body.

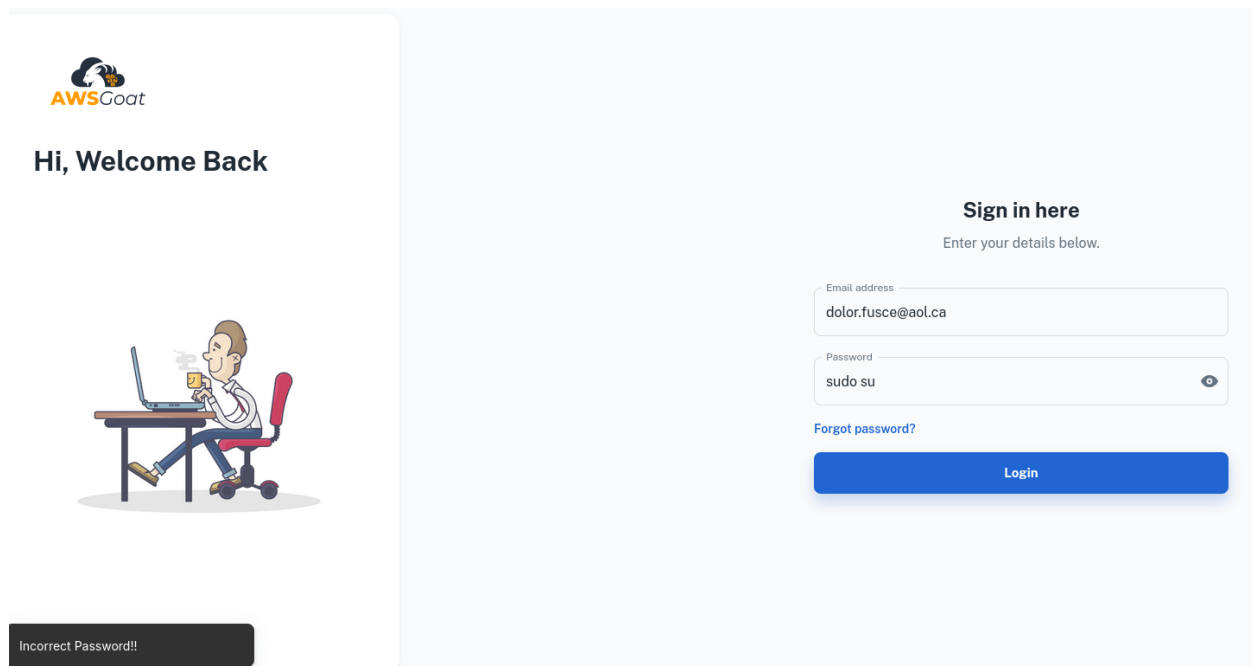


Figura 6: acceso fallido

3.4 Exposición de datos sensibles en /dump

Problema identificado

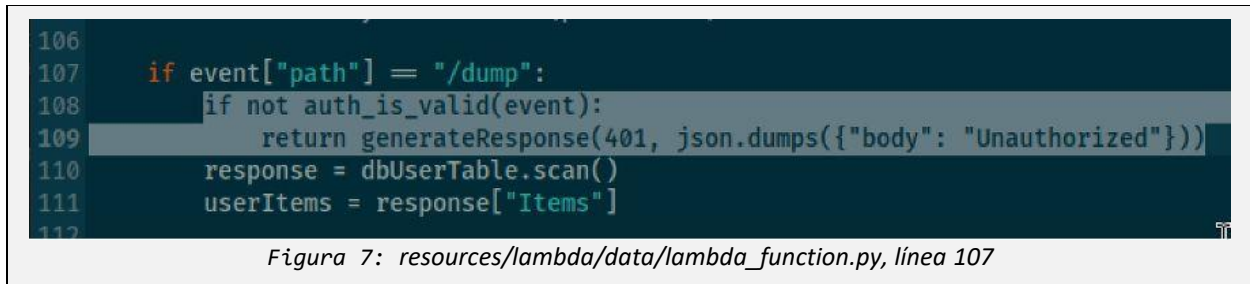
El endpoint /dump ejecutaba un escaneo completo de la tabla de usuarios (dbUserTable.scan()) y devolvía su contenido íntegro sin exigir ningún tipo de autenticación, exponiendo credenciales y datos personales de todos los usuarios registrados a cualquier visitante no autenticado.

Antes

```
if event["path"] == "/dump":  
    response = dbUserTable.scan()
```

Después

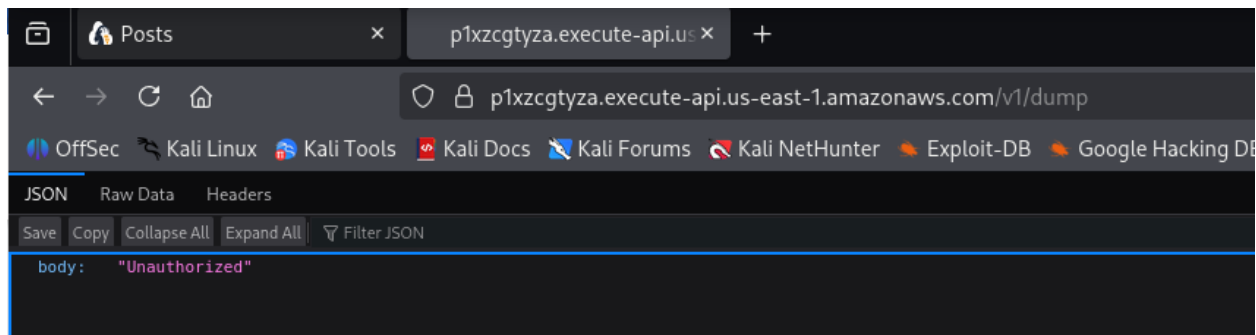
```
if event["path"] == "/dump":  
    if not auth_is_valid(event):  
        return generateResponse(401,  
                                json.dumps({"body": "Unauthorized"}))  
  
    response = dbUserTable.scan()
```



Se añadió la misma validación de sesión (auth_is_valid) utilizada en el reto anterior, exigiendo un JWT válido antes de ejecutar el escaneo de la tabla. El endpoint deja de ser accesible de forma anónima.

Verificación

Se accedió directamente al endpoint /dump desde una ventana de navegación privada, sin sesión iniciada. Tras el fix, la respuesta es 401 Unauthorized en lugar del volcado JSON completo de la tabla de usuarios.



3.5 Hallazgo adicional: deficiencias de Terraform que impedían la propagación de los parches

Problema identificado

Durante la fase de remediación se detectó que, si bien el código fuente corregido (React y Python) era sintácticamente correcto y coincidía con lo indicado en la guía de fixes, un terraform apply posterior a la edición no actualizaba los recursos desplegados en AWS: el plan reportaba "No changes" pese a que el contenido de los archivos había cambiado. La causa raíz, común a los tres componentes afectados, fue la ausencia de mecanismos de Terraform para vincular el contenido de un archivo local al estado del recurso remoto.

Componente	Deficiencia detectada
aws_lambda_function.lambda_ba_data	No declaraba source_code_hash. Terraform no vinculaba el contenido del paquete .zip regenerado por data.archive_file al estado del recurso, por lo que un terraform apply posterior a un cambio de código no actualizaba la función en AWS ("No changes" aun con el .py modificado).
aws_s3_object (upload_folder_prod, upload_folder_dev, upload_temp_object, etc.)	No declaraban etag. Terraform no detectaba cambios de contenido en archivos cuyo nombre (key) permanecía igual, por lo que un frontend recompilado no se volvía a subir a S3 salvo que se forzara manualmente.

null_resource.file_replacement_api_gw / file_replacement_lambda_data / file_replacement_lambda_react	provisioner "local-exec" sin bloque triggers. Estos recursos solo se ejecutan en la creación inicial; en aplicaciones posteriores del mismo entorno, los reemplazos de placeholders (API_GATEWAY_URL, nombre de bucket) no vuelven a correr sobre archivos regenerados manualmente.
---	---

Se corrigió agregando el atributo `source_code_hash` = `data.archive_file.lambda_zip_bap.output_base64sha256` al recurso `aws_lambda_function.lambda_ba_data`, y el atributo `etag` = `filemd5("<ruta>/${each.value}")` a cada `aws_s3_object` que sube el build del frontend (`upload_folder_prod`, `upload_folder_dev`, `upload_temp_object`). Con estos cambios, Terraform compara el hash del contenido en cada plan/apply y detecta correctamente cuándo el código fuente o el build fue modificado, sin necesidad de forzar reemplazos manuales.

Verificación

Tras agregar `source_code_hash`, terraform plan mostró la actualización en línea (~ update in-place) del recurso `aws_lambda_function.lambda_ba_data`, y `aws lambda get-function` confirmó, mediante comparación del campo `CodeSha256` contra el hash del paquete local, que el código efectivamente desplegado en AWS correspondía al código fuente corregido. De forma análoga, tras agregar `etag` a los `aws_s3_object` del frontend, terraform plan detectó y aplicó la actualización del archivo `main.*.js` modificado, resolviendo además un efecto secundario (los provisioners `local-exec` de reemplazo de URL de API Gateway, que tampoco se re-ejecutan en aplicaciones posteriores por carecer de un bloque triggers, debieron correrse manualmente sobre el build recompilado para restaurar las rutas absolutas de API Gateway y del bucket S3).

PARTE II – MÓDULO 2: APLICACIÓN HR

Informe Técnico de Blue Team — Infraestructura ECS, RDS, IAM y monitoreo (CloudTrail/CloudWatch)

4. Arquitectura desplegada

La infraestructura del Módulo 2 fue desplegada mediante Terraform sobre una única cuenta de AWS, región us-east-1. Los servicios efectivamente desplegados y verificados durante el ejercicio son los siguientes:

- Red: una VPC (10.0.0.0/16) con dos subredes públicas, Internet Gateway y tabla de rutas pública.
- Cómputo: clúster de Amazon ECS (ecs-lab-cluster) sobre instancias EC2 administradas por un Auto Scaling Group y un Launch Template, con el contenedor de la aplicación HR (aws-goat-m2).
- Balanceo de carga: un Application Load Balancer (aws-goat-m2-alb) con su Target Group y Listener en el puerto 80.
- Base de datos: una instancia RDS MySQL (aws-goat-db) en un subnet group dedicado, con Security Group propio que solo permite tráfico en el puerto 3306 desde el Security Group de ECS.
- Gestión de secretos: AWS Secrets Manager, con el secreto RDS_CREDS que almacena las credenciales de la base de datos.
- Identidad y acceso (IAM): roles ecs-instance-role, ecs-task-role y ec2-deployer-role, cada uno con políticas administradas y personalizadas, además de una permissions boundary aplicada al rol de instancia.
- Almacenamiento: dos buckets de Amazon S3 — do-not-delete-awsgoat-state-files-501578625392, utilizado para el estado de Terraform, y aws-goat-m2-logs-501578625392, utilizado como repositorio centralizado de logs (CloudTrail, ALB y accesos al bucket de estado).
- Auditoría: AWS CloudTrail (aws-goat-m2-trail), con entrega de eventos tanto a S3 como a un log group de CloudWatch Logs.
- Monitoreo y alertas: grupos de log de CloudWatch (/ecs/aws-goat-m2 y aws-cloudtrail-logs), un metric filter sobre eventos de IAM, dos alarmas de CloudWatch (cambios de IAM y objetivos no saludables del ALB) y un tema de Amazon SNS (aws-goat-m2-security-alerts) con una suscripción por correo electrónico.

No se registró evidencia en este ejercicio del uso de AWS Lambda, API Gateway o WAF dentro del Módulo 2; por lo tanto, no se documentan controles sobre dichos servicios en este informe.

5. Vulnerabilidades identificadas

A partir de la comparación entre la configuración original del archivo main.tf del Módulo 2 y las prácticas recomendadas de seguridad en AWS, se identificaron las siguientes debilidades de configuración. Estas se describen desde la perspectiva de detección y análisis del Blue Team; no se detalla el procedimiento de explotación asociado.

Vulnerabilidad	Servicio	Riesgo asociado	OWASP	Buena práctica AWS relacionada
Rol de instancia ECS con política administrada IAMFullAccess adjunta	IAM (ecs-instance-role)	Alto. Cualquier proceso con acceso a la instancia podía crear	A01:2021 – Broken Access	Principio de mínimo privilegio (AWS Well-Architected, Security Pillar)

Vulnerabilidad	Servicio	Riesgo asociado	OWASP	Buena práctica AWS relacionada
		usuarios, políticas y roles arbitrarios, escalando privilegios sobre toda la cuenta.	Control	
Política de permissions boundary del rol de instancia con iam:PassRole e iam:PutRole* sin restricción de recurso	IAM (instance_boundary_policy)	Alto. Permitía modificar roles existentes y pasar cualquier rol de la cuenta a cualquier servicio, habilitando escalamiento de privilegios.	A01:2021 – Broken Access Control	Las boundary policies deben acotar el alcance real de los permisos, no solo delegarlos
Rol ec2-deployer con política administrativa total (Action "*", Resource "**")	IAM (ec2-deployer-role)	Crítico. Cualquier entidad que asumiera este rol obtenía control total sobre la cuenta de AWS.	A01:2021 / A05:2021 – Broken Access Control / Security Misconfiguration	Least privilege access; evitar políticas comodín en producción
Rol de tarea ECS con la política gestionada SecretsManagerReadWrite	IAM (ecs-task-role) / Secrets Manager	Medio-alto. La tarea podía leer y escribir cualquier secreto de la cuenta, no solo las credenciales de RDS que requiere.	A05:2021 – Security Misconfiguration	Los permisos deben acotarse al recurso (ARN) específico que la carga de trabajo necesita
Bucket S3 de estado de Terraform sin bloqueo de acceso público, cifrado, versionado ni registro de acceso	S3 (bucket_tf_files)	Alto. El archivo de estado contiene información sensible de la infraestructura; su exposición o alteración compromete confidencialidad e integridad del entorno.	A05:2021 – Security Misconfiguration	Cifrado en reposo, bloqueo de acceso público y trazabilidad de acceso en todo bucket
Ausencia de registro de auditoría (CloudTrail) y de monitoreo activo (CloudWatch) sobre eventos de IAM y disponibilidad del servicio	CloudTrail / CloudWatch	Alto. Sin trazabilidad de eventos de la cuenta ni alertas, cambios no autorizados o caídas del servicio podían pasar desapercibidos.	A09:2021 – Security Logging and Monitoring Failures	Registro y monitoreo continuo como control fundamental del pilar de seguridad de AWS
Application Load Balancer sin registro de logs de acceso	ALB (aws-goat-m2-alb)	Medio. Imposibilita el análisis forense del tráfico HTTP recibido en caso de incidente.	A09:2021 – Security Logging and Monitoring Failures	Habilitar access logs en todo balanceador expuesto a internet

6. Controles y remediaciones aplicadas

Para cada vulnerabilidad identificada se modificó el código Terraform correspondiente. A continuación se presentan los extractos relevantes del archivo main.tf en su versión original y en su versión corregida, la explicación del problema, la acción realizada, el beneficio de seguridad obtenido y, cuando existe evidencia, las capturas que documentan su verificación.

6.1 Reducción de privilegios en el rol de instancia ECS

Problema identificado

El rol ecs-instance-role tenía adjunta la política administrada IAMFullAccess, otorgando control total sobre el servicio IAM a cualquier proceso que se ejecutara sobre la instancia.

Antes

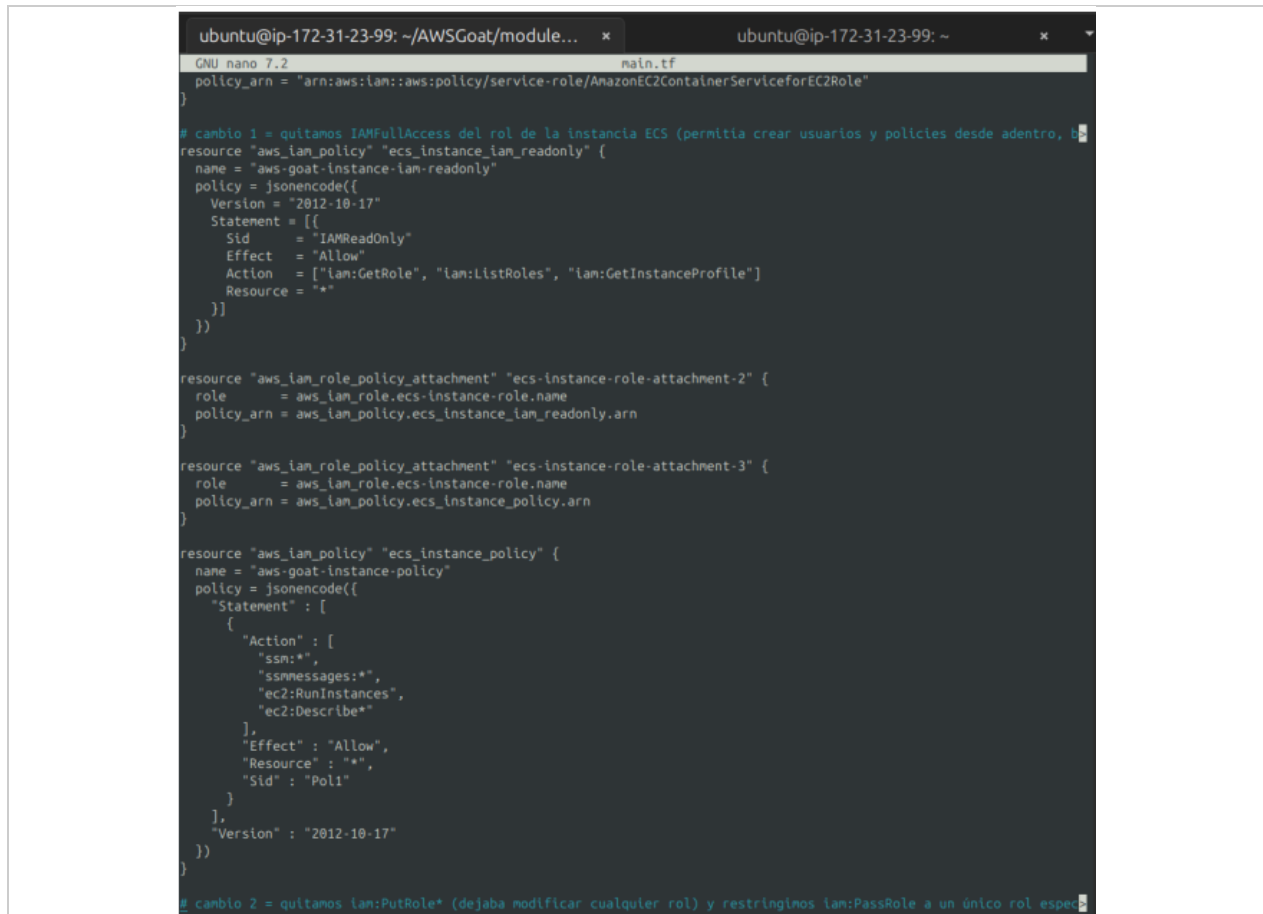
```
resource "aws_iam_role_policy_attachment" "ecs-instance-role-attachment-2" {
  role      = aws_iam_role.ecs-instance-role.name
  policy_arn = "arn:aws:iam::aws:policy/IAMFullAccess"
}
```

Después

```
resource "aws_iam_policy" "ecs_instance_iam_readonly" {
  name = "aws-goat-instance-iam-readonly"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid      = "IAMReadOnly"
      Effect   = "Allow"
      Action   = ["iam:GetRole", "iam:ListRoles", "iam:GetInstanceProfile"]
      Resource = "*"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ecs-instance-role-attachment-2" {
  role      = aws_iam_role.ecs-instance-role.name
  policy_arn = aws_iam_policy.ecs_instance_iam_readonly.arn
}
```

Se sustituyó una política administrada de control total sobre IAM por una política personalizada de solo lectura, limitada a tres acciones de consulta. Esto elimina la posibilidad de que la instancia cree, modifique o elimine usuarios, roles o políticas, conservando únicamente la capacidad de consulta necesaria para el funcionamiento del laboratorio.



```
ubuntu@ip-172-31-23-99: ~/AWSGoat/module... x ubuntu@ip-172-31-23-99: ~ x
GNU nano 7.2 main.tf
policy_arn = "arn:aws:iam::aws:policy/service-role/AmazonEC2ContainerServiceforEC2Role"
}

# cambio 1 = quitamos IAMFullAccess del rol de la instancia ECS (permitia crear usuarios y policies desde adentro, b
resource "aws_iam_policy" "ecs_instance_iam_readonly" {
  name = "aws-goat-instance-iam-readonly"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid = "IAMReadOnly"
      Effect = "Allow"
      Action = ["iam:GetRole", "iam:ListRoles", "iam:GetInstanceProfile"]
      Resource = "*"
    }]
  })
}

resource "aws_iam_role_policy_attachment" "ecs-instance-role-attachment-2" {
  role = aws_iam_role.ecs-instance-role.name
  policy_arn = aws_iam_policy.ecs_instance_iam_readonly.arn
}

resource "aws_iam_role_policy_attachment" "ecs-instance-role-attachment-3" {
  role = aws_iam_role.ecs-instance-role.name
  policy_arn = aws_iam_policy.ecs_instance_policy.arn
}

resource "aws_iam_policy" "ecs_instance_policy" {
  name = "aws-goat-instance-policy"
  policy = jsonencode({
    "Statement" : [
      {
        "Action" : [
          "ssm:*",
          "ssmmessages:*",
          "ec2:RunInstances",
          "ec2:Describe*"
        ],
        "Effect" : "Allow",
        "Resource" : "*",
        "Sid" : "Pol1"
      }
    ],
    "Version" : "2012-10-17"
  })
}

# cambio 2 = quitamos iam:PutRole* (dejaba modificar cualquier rol) y restringimos iam:PassRole a un único rol espe
```

Figura 1. Edición del archivo main.tf en el editor nano, mostrando la eliminación de IAMFullAccess (cambio 1) y el ajuste de la permissions boundary (cambio 2) aplicados directamente sobre el entorno (GUI/terminal).

6.2 Restricción de la permissions boundary del rol de instancia

Problema identificado

La política de permissions boundary asociada al rol de instancia incluía las acciones iam:PassRole e iam:PutRole* sin restricción de recurso, lo que permitía pasar cualquier rol de la cuenta a cualquier servicio y modificar roles existentes, habilitando un posible escalamiento de privilegios.

Antes

```
resource "aws_iam_policy" "instance_boundary_policy" {
  name = "aws-goat-instance-boundary-policy"
  policy = jsonencode({
    "Statement" : [{
      "Action" : [
        "iam:List*", "iam:Get*", "iam:PassRole", "iam:PutRole*",
        "ssm:*", "ssmmessages:*", "ec2:RunInstances", "ec2:Describe*",
        "ecs:*", "ecr:*", "logs:CreateLogStream", "logs:PutLogEvents"
      ],
      "Effect" : "Allow",
      "Resource" : "*",
      "Sid" : "Pol1"
    }],
    "Version" : "2012-10-17"
  })
}
```



```

    })
  }
}

```

Después

```

resource "aws_iam_policy" "instance_boundary_policy" {
  name = "aws-goat-instance-boundary-policy"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      { Sid = "LimitedIAM", Effect = "Allow",
        Action = ["iam:List*", "iam:Get*"], Resource = "*" },
      { Sid = "RestrictedPassRole", Effect = "Allow",
        Action = "iam:PassRole",
        Resource = "arn:aws:iam::<cuenta>:role/ecs-instance-role",
        Condition = { StringEquals = {
          "iam:PassedToService" = "ec2.amazonaws.com" } } },
      { Sid = "OperationalPermissions", Effect = "Allow",
        Action = ["ssm:GetParameter*", "ssmmessages:*",
          "ec2:RunInstances", "ec2:Describe*", "ecs:*", "ecr:*",
          "logs:CreateLogStream", "logs:PutLogEvents"],
        Resource = "*" }
    ]
  })
}

```

Se eliminó la acción `iam:PutRole*` (que permitía modificar cualquier rol) y se restringió `iam:PassRole` a un único rol específico (`ecs-instance-role`), condicionado a que el servicio destino sea EC2. Con ello se reduce la superficie de escalamiento de privilegios sin afectar la operación normal de la instancia. Tras aplicar estos cambios se ejecutó `terraform init -upgrade` y `terraform plan` sobre el directorio del Módulo 2, confirmando que el código corregido inicializa y planifica correctamente.

```

ubuntu@ip-172-31-23-99:~/AWSGoat/modules/module-2$ terraform init -upgrade
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/null...
- Finding latest version of hashicorp/template...
- Finding hashicorp/aws versions matching "~> 3.75"...
- Using previously-installed hashicorp/null v3.3.0
- Using previously-installed hashicorp/template v2.2.0
- Using previously-installed hashicorp/aws v3.76.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-23-99:~/AWSGoat/modules/module-2$ terraform plan
data.template_file.user_data: Reading...
data.template_file.user_data: Read complete after 0s [id=79be124c616c2981b5d9561dc217
5e65c60bc696f1723d4f40e3403b48f9a2e5]
aws_iam_role.ecs-task-role: Refreshing state... [id=ecs-task-role]

```

Figura 2. Verificación por CLI: `terraform init -upgrade` finalizado con éxito y `terraform plan` en ejecución sobre el directorio `modules/module-2` tras aplicar las correcciones de IAM.

6.3 Eliminación de permisos administrativos totales en el rol `ec2-deployer`

Problema identificado

La política `ec2DeployerAdmin-policy` otorgaba la acción `"*"` sobre el recurso `"*"`, es decir, permisos administrativos completos sobre toda la cuenta de AWS a cualquier entidad que asumiera el rol `ec2-deployer-role`.

Antes

```
resource "aws_iam_policy" "ec2_deployer_admin_policy" {
  name = "ec2DeployerAdmin-policy"
  policy = jsonencode({
    "Statement" : [{ "Action" : ["*"], "Effect" : "Allow",
      "Resource" : "*", "Sid" : "Policy1" }],
    "Version" : "2012-10-17"
  })
}
```

Después

```
resource "aws_iam_policy" "ec2_deployer_admin_policy" {
  name = "ec2DeployerAdmin-policy"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Sid = "EC2DeployOnly"
      Effect = "Allow"
      Action = ["ec2:RunInstances", "ec2:TerminateInstances",
        "ec2:DescribeInstances", "ec2:CreateTags"]
      Resource = "*"
    }]
  })
}
```

Se acotó el rol a únicamente las acciones de despliegue de instancias EC2 que su función requiere, eliminando el acceso administrativo total sobre la cuenta. No se cuenta con una captura específica de la validación de este recurso más allá de su presencia confirmada en el inventario de recursos etiquetados (ver sección 4.13, Figura sobre AWS Resource Groups), por lo que no se documenta una prueba funcional adicional para este control.

6.4 Restricción del rol de tarea ECS sobre Secrets Manager

Problema identificado

El rol `ecs-task-role` tenía adjunta la política administrada `SecretsManagerReadWrite`, que otorga lectura y escritura sobre todos los secretos de la cuenta, cuando la tarea únicamente necesita leer las credenciales de la base de datos RDS.

Antes

```
resource "aws_iam_role_policy_attachment" "ecs-task-role-attachment-2" {
  role      = aws_iam_role.ecs-task-role.name
  policy_arn = "arn:aws:iam::aws:policy/SecretsManagerReadWrite"
}
```

Después

```
resource "aws_iam_policy" "ecs_task_secrets_readonly" {
```

```

name = "aws-goat-task-secrets-readonly"
policy = jsonencode({
  Version = "2012-10-17"
  Statement = [{
    Sid      = "ReadOnlyRDSecret"
    Effect   = "Allow"
    Action   = "secretsmanager:GetSecretValue"
    Resource = aws_secretsmanager_secret.rds_creds.arn
  }]
})
}

```

La tarea ECS queda restringida a la acción GetSecretValue sobre el ARN específico del secreto RDS_CREDS, sin capacidad de escritura ni de lectura sobre otros secretos de la cuenta. No existe evidencia adicional (CLI o consola) más allá del código y de la existencia del recurso en el inventario de etiquetado; no se documenta una prueba funcional adicional para este control.

6.5 Cifrado, bloqueo de acceso público, versionado y registro de acceso en el bucket de estado de Terraform

Problema identificado

El bucket `bucket_tf_files`, que almacena el estado de Terraform (con información sensible de la infraestructura), no contaba con bloqueo de acceso público, cifrado en reposo, versionado ni registro de acceso.

Antes

```

resource "aws_s3_bucket" "bucket_tf_files" {
  bucket        = "do-not-delete-awsgoat-state-files-<cuenta>"
  force_destroy = true
  tags = { Name = "Do not delete Bucket", Environment = "Dev" }
}

```

Después (añadido sobre el mismo bucket)

```

resource "aws_s3_bucket_public_access_block" "bucket_tf_files_pab" {
  bucket = aws_s3_bucket.bucket_tf_files.id
  block_public_acls = true
  block_public_policy = true
  ignore_public_acls = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_server_side_encryption_configuration" "bucket_tf_files_enc" {
  bucket = aws_s3_bucket.bucket_tf_files.id
  rule { apply_server_side_encryption_by_default { sse_algorithm = "AES256" } }
}

resource "aws_s3_bucket_versioning" "bucket_tf_files_ver" {
  bucket = aws_s3_bucket.bucket_tf_files.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_logging" "bucket_tf_files_logging" {
  bucket          = aws_s3_bucket.bucket_tf_files.id
  target_bucket   = aws_s3_bucket.access_logs_bucket.id
  target_prefix   = "tfstate-bucket-logs/"
}

```

```
}
```

Se añadieron cuatro controles independientes sobre el bucket de estado: bloqueo total de acceso público, cifrado en reposo con AES-256, versionado y registro de acceso hacia un bucket dedicado. La consola de S3 confirma, en la sección "Registro de acceso del servidor" del bucket `do-not-delete-aws-goat-state-files-501578625392`, que el registro está habilitado con destino al bucket `aws-goat-m2-logs-501578625392`.

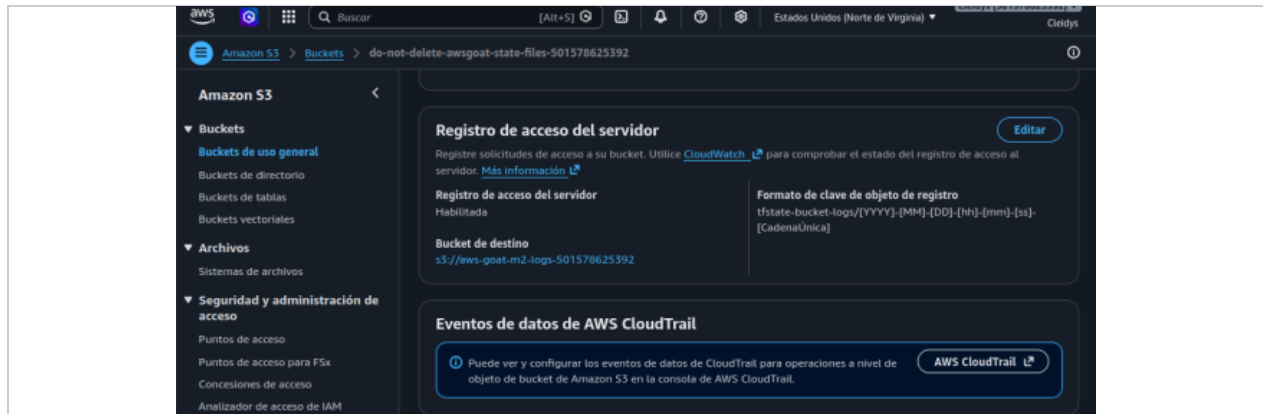


Figura 3. Consola de Amazon S3: registro de acceso del servidor habilitado sobre el bucket de estado de Terraform, con bucket de destino `aws-goat-m2-logs-501578625392` (GUI).

6.6 Bucket dedicado de logs y protección de acceso público

Problema identificado

No existía un repositorio centralizado y protegido para recibir los logs de CloudTrail, del ALB y del propio bucket de estado.

Acción realizada

Se creó el bucket `aws-goat-m2-logs-501578625392`, con su propio bloqueo de acceso público (`aws_s3_bucket_public_access_block`) y una política de bucket (`aws_s3_bucket_policy`) que otorga permisos de escritura exclusivamente a los servicios de CloudTrail y al ALB, sobre los prefijos `cloudtrail/` y `alb-logs/` respectivamente.

La verificación mediante AWS CLI confirmó que las cuatro banderas de bloqueo de acceso público se encuentran habilitadas sobre este bucket.

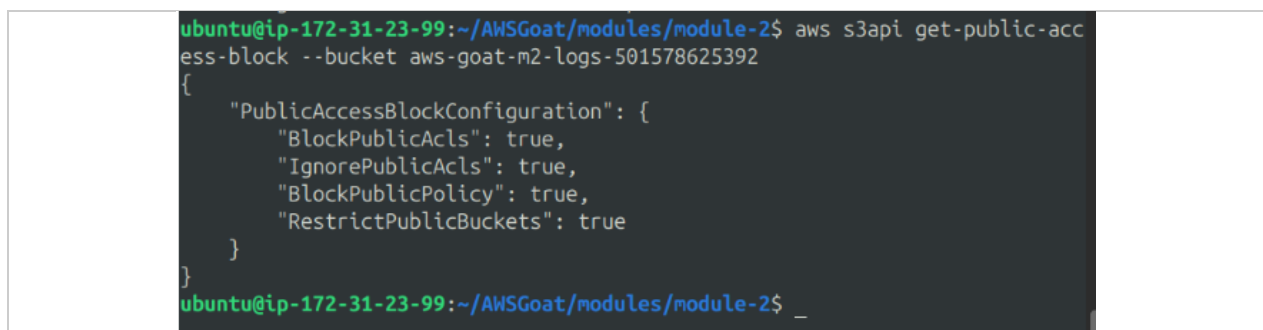


Figura 4. Verificación por AWS CLI (`aws s3api get-public-access-block`) del bloqueo de acceso público sobre el bucket `aws-goat-m2-logs-501578625392`: las cuatro banderas en `true`.

6.7 Habilitación de registro de acceso del Application Load Balancer

Problema identificado

El Application Load Balancer `aws-goat-m2-alb` no tenía habilitado el registro de logs de acceso, lo que impedía el análisis forense del tráfico HTTP recibido.

Antes

```
resource "aws_alb" "application_load_balancer" {
  name           = "aws-goat-m2-alb"
  internal       = false
  load_balancer_type = "application"
  subnets       = [ ... ]
  security_groups = [ ... ]
  tags = { Name = "aws-goat-m2-alb" }
}
```

Después

```
resource "aws_alb" "application_load_balancer" {
  name           = "aws-goat-m2-alb"
  internal       = false
  load_balancer_type = "application"
  subnets       = [ ... ]
  security_groups = [ ... ]

  access_logs {
    bucket = aws_s3_bucket.access_logs_bucket.id
    prefix = "alb-logs"
    enabled = true
  }

  tags = { Name = "aws-goat-m2-alb" }
  depends_on = [aws_s3_bucket_policy.cloudtrail_bucket_policy]
}
```

Se habilitó el registro de logs de acceso del ALB hacia el bucket de logs dedicado. La verificación se realizó por dos vías: por AWS CLI, confirmando que el atributo `access_logs.s3.enabled` está en `true` y apunta al bucket `aws-goat-m2-logs-501578625392`; y por consola, comprobando en Amazon S3 la existencia de archivos de log reales generados por el balanceador (objetos bajo el prefijo `elasticloadbalancing`) junto con el archivo de prueba `ELBAccessLogTestFile` que AWS genera al habilitar la función.

```
ubuntu@ip-172-31-23-99:~/AWSGoat/modules/module-2$ aws elbv2 describe-load-balancer-attributes --load-balancer-arn $(aws elbv2 describe-load-balancers --names aws-goat-m2-alb --query "LoadBalancers[0].LoadBalancerArn" --output text)
{
  "Attributes": [
    {
      "Key": "access_logs.s3.enabled",
      "Value": "true"
    },
    {
      "Key": "access_logs.s3.bucket",
      "Value": "aws-goat-m2-logs-501578625392"
    },
  ],
}
```

Figura 5. Verificación por AWS CLI (`aws elbv2 describe-load-balancer-attributes`) de los atributos `access_logs.s3.enabled = true` y `access_logs.s3.bucket = aws-goat-m2-logs-501578625392`.

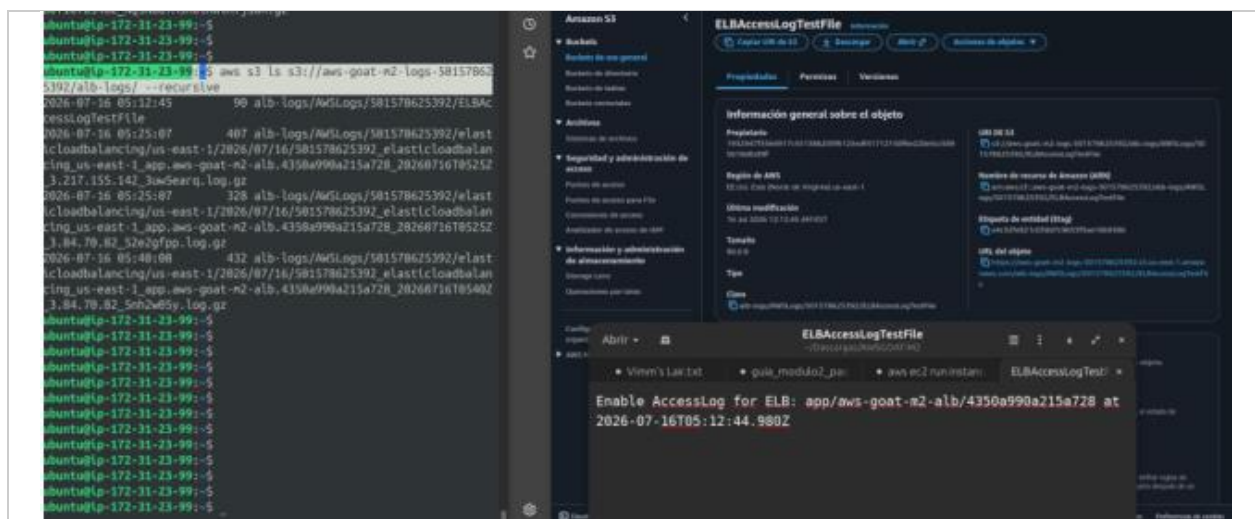


Figura 6. Verificación por consola/CLI en Amazon S3: objetos de log reales generados por el ALB (prefijo elasticloadbalancing) y archivo ELBAccessLogTestFile, confirmando que el registro está operativo (GUI).

6.8 Implementación de CloudTrail con entrega a S3 y CloudWatch Logs

Problema identificado

La infraestructura original no contaba con un mecanismo de auditoría (CloudTrail) que registrara las llamadas a la API de AWS realizadas dentro de la cuenta.

Acción realizada

Se implementó el trail `aws-goat-m2-trail`, con entrega de eventos al bucket de logs dedicado (prefijo `cloudtrail/`) y, adicionalmente, a un log group de CloudWatch Logs (`aws-cloudtrail-logs`), para lo cual se creó un rol IAM (`cloudtrail_to_cwlogs_role`) con permisos mínimos limitados a `logs:CreateLogStream` y `logs:PutLogEvents` sobre ese log group específico. Se habilitó la validación de integridad de los archivos de log (`enable_log_file_validation`).

La consola de CloudTrail confirma el trail `aws-goat-m2-trail` en estado de registro activo.

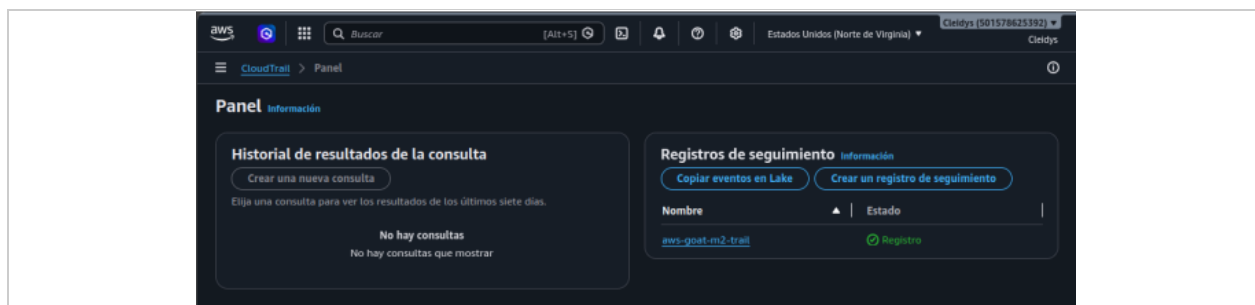


Figura 7. Consola de AWS CloudTrail: trail `aws-goat-m2-trail` en estado "Registro" activo (GUI).

El historial de eventos de CloudTrail muestra, a modo de ejemplo, el evento `ListRoles` ejecutado por el usuario `awsgoat-deploy`, confirmando que la auditoría de llamadas a la API de IAM se encuentra operativa.

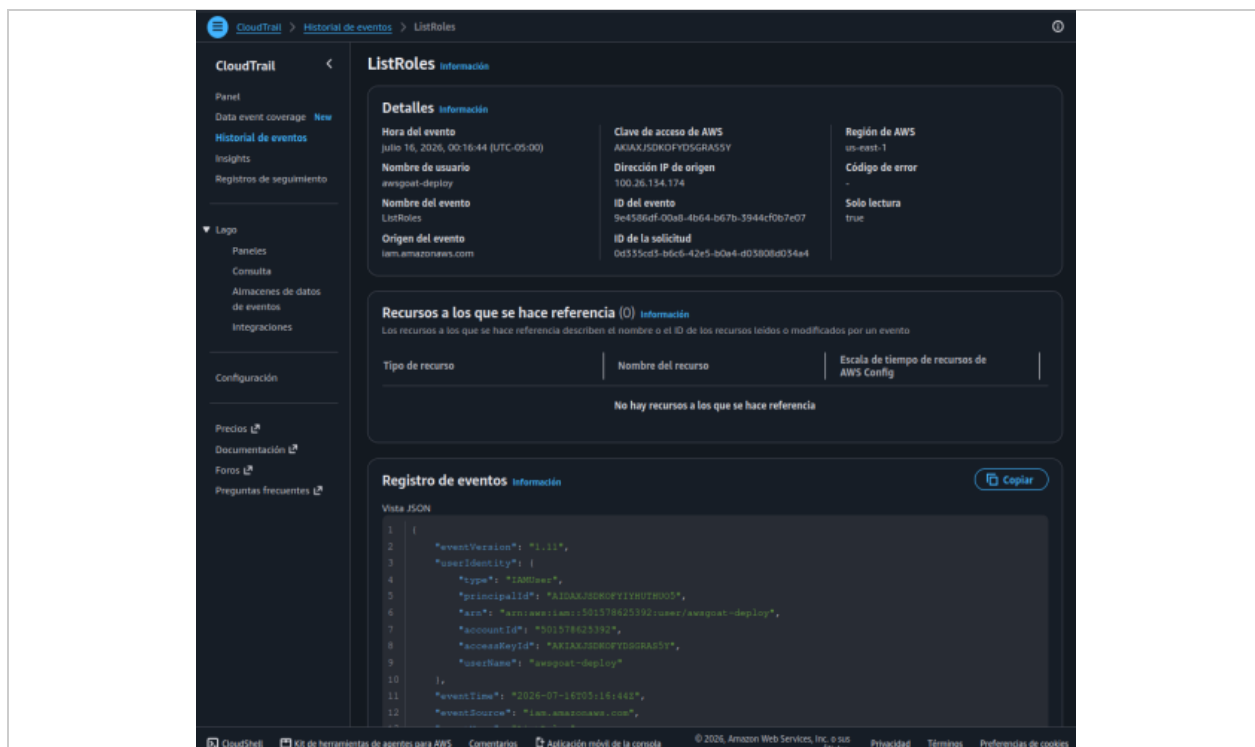


Figura 8. Consola de CloudTrail — Historial de eventos: evento ListRoles registrado, con identidad del usuario awsgoat-deploy y detalle del evento en formato JSON (GUI).

6.9 Detección de cambios de IAM mediante Metric Filter y alarma

Problema identificado

Aun con CloudTrail activo, no existía un mecanismo de notificación automática ante la creación de usuarios o políticas de IAM, lo que retrasaría la detección de actividad no autorizada.

Acción realizada

Sobre el log group aws-cloudtrail-logs se configuró un metric filter (IAMPolicyOrUserChanges) que detecta los eventos CreateUser, AttachUserPolicy y CreatePolicy, generando la métrica personalizada IAMChangesCount en el namespace AWSGoatSecurity. Se configuró la alarma aws-goat-m2-iam-changes, que se activa cuando dicha métrica es mayor a cero en un período de 300 segundos, notificando al tema de SNS aws-goat-m2-security-alerts.

El control fue validado de forma activa mediante una prueba dirigida por AWS CLI: se creó un usuario de prueba (test-alarma-defensa).

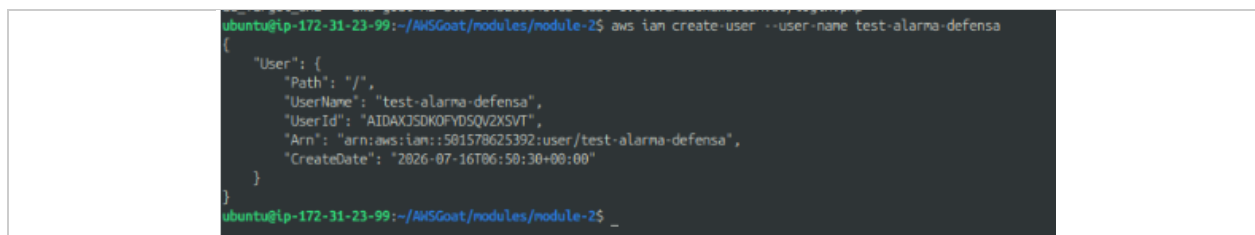


Figura 9. Prueba dirigida por AWS CLI: creación del usuario IAM test-alarma-defensa (aws iam create-user) para verificar la detección de cambios de IAM.

El evento quedó registrado y filtrado correctamente en CloudWatch Logs, dentro del log group alimentado por CloudTrail:

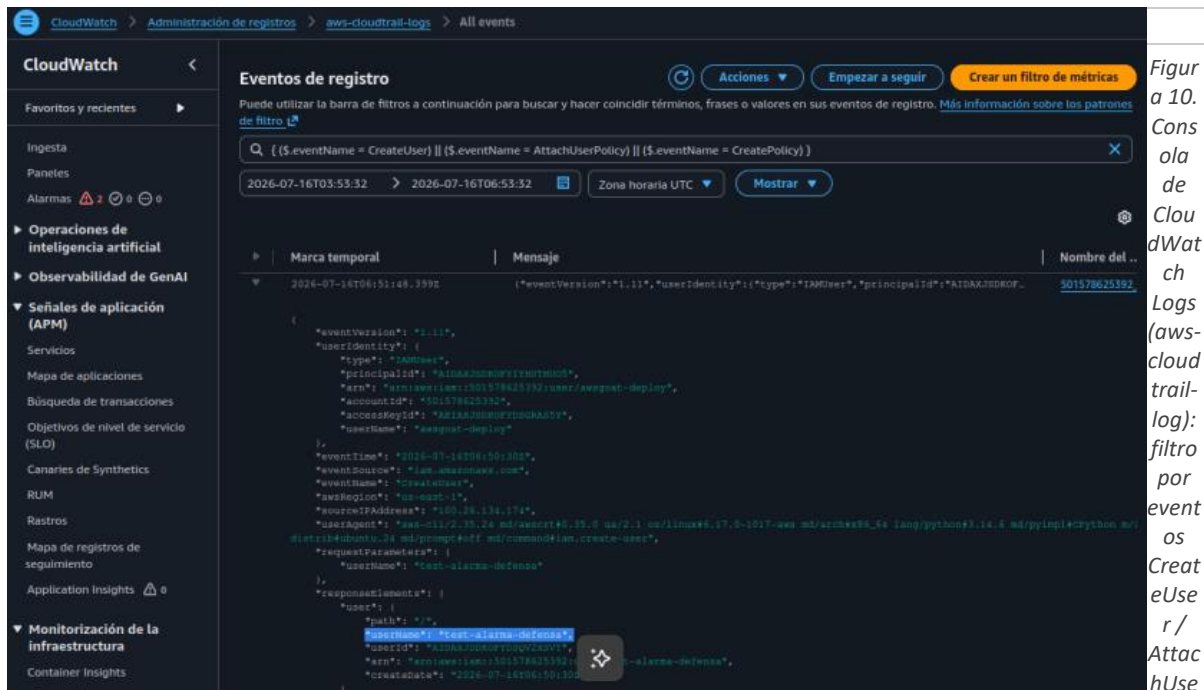


Figura 10. Consola de CloudWatch Logs (aws-cloudtrail-log): filtro por evento `CreateUser/AttachUserPolicy/`, mostrando el evento de creación del usuario `test-alarma-defensa` por el usuario `aws-goat-deploy` (GUI).

La alarma correspondiente pasó a estado de alarma en la consola de CloudWatch:

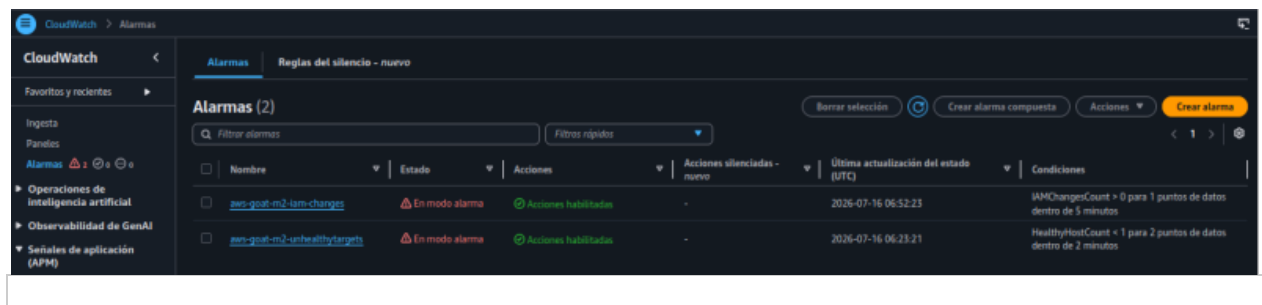
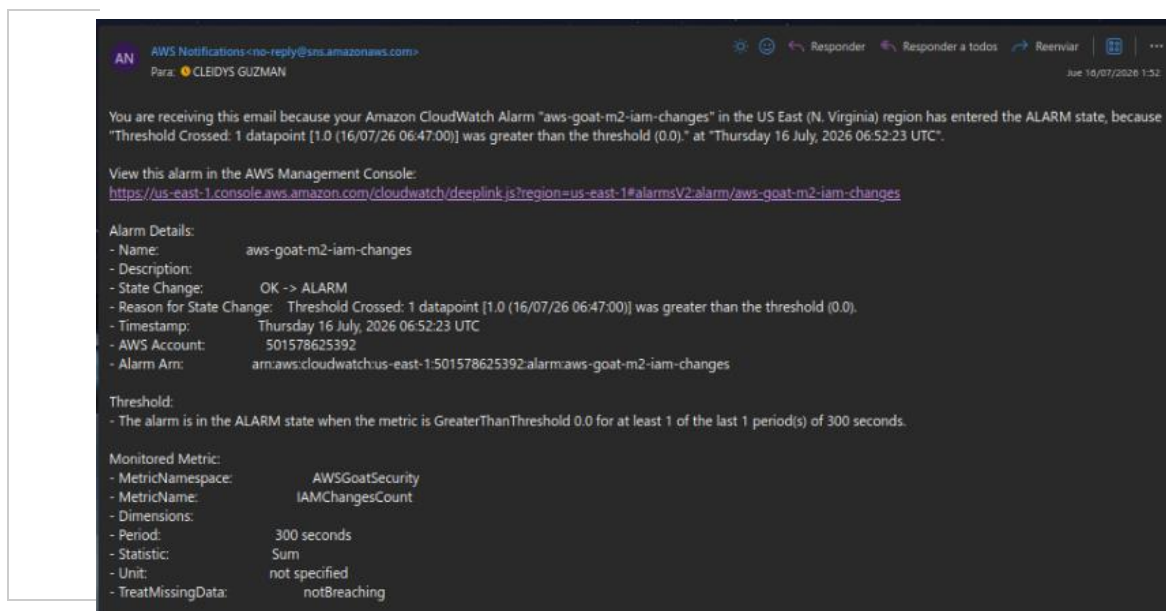


Figura 11. Consola de CloudWatch — Alarmas: `aws-goat-m2-iam-changes` y `aws-goat-m2-unhealthytargets`, ambas en estado "En alarma", con la hora de la última actualización (GUI).

Y se recibió la notificación correspondiente por correo electrónico a través de Amazon SNS:



Figura

12. Notificación por correo electrónico (Amazon SNS) del cambio de estado de la alarma aws-goat-m2-iam-changes, con el detalle del umbral y la métrica monitoreada.

Esta prueba dirigida demuestra que la cadena completa de detección — CloudTrail, CloudWatch Logs, metric filter, alarma y notificación — funciona de extremo a extremo.

6.10 Monitoreo de disponibilidad del servicio (ALB / ECS)

Problema identificado

No existía una alarma que notificara la pérdida de objetivos saludables detrás del Application Load Balancer, lo que retrasaría la detección de una caída del servicio.

Acción realizada

Se configuró la alarma aws-goat-m2-unhealthytargets sobre la métrica HealthyHostCount del Target Group del ALB, que se activa cuando el conteo de hosts saludables es menor a 1 durante dos períodos consecutivos de 60 segundos, notificando también al tema de SNS aws-goat-m2-security-alerts.

El control fue validado de forma activa: se llevó el servicio ECS (ecs_service_worker) a desired-count 0 mediante AWS CLI, simulando la pérdida de instancias saludables detrás del balanceador.

```
ubuntu@ip-172-31-23-99:~/AWSGoat/modules/module-2$ aws ecs update-service --cluster
ecs-lab-cluster --service ecs_service_worker --desired-count 0
{
  "service": {
    "serviceArn": "arn:aws:ecs:us-east-1:501578625392:service/ecs-lab-cluster
/ecs_service_worker",
    "serviceName": "ecs_service_worker",
    "clusterArn": "arn:aws:ecs:us-east-1:501578625392:cluster/ecs-lab-cluster
",
    "loadBalancers": [
      {
        "targetGroupArn": "arn:aws:elasticloadbalancing:us-east-1:5015786
25392:targetgroup/aws-goat-m2-tg/7e9cb7bc257132e3",
        "containerName": "aws-goat-m2",
        "containerPort": 80
      }
    ],
    "serviceRegistries": [],
    "status": "ACTIVE",
    "desiredCount": 0,
    "runningCount": 1,
    "pendingCount": 0,
    "launchType": "EC2",
    "taskDefinition": "arn:aws:ecs:us-east-1:501578625392:task-definition/ECS
-Lab-Task-definition:5",
  }
}
```

Figura 13. Prueba dirigida por AWS CLI (aws ecs update-service --desired-count 0): reducción del servicio ecs_service_worker a cero tareas deseadas para simular la pérdida de disponibilidad.

La alarma pasó a estado de alarma en la consola de CloudWatch (ver Figura de la sección 4.9, donde se observan ambas alarmas activas), y se confirmó la notificación correspondiente por correo electrónico:

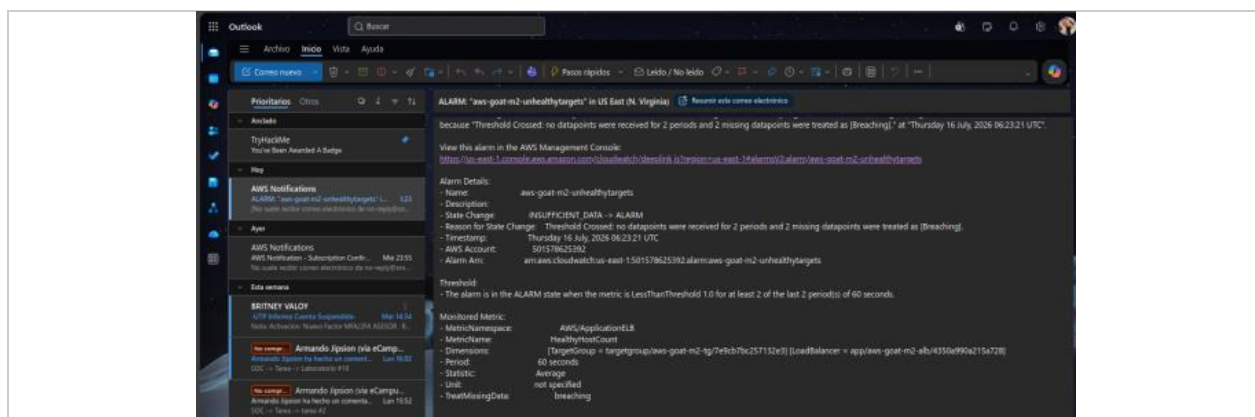


Figura 14. Notificación por correo electrónico de la alarma aws-goat-m2-unhealthytargets tras la prueba dirigida, indicando el cruce de umbral en la métrica HealthyHostCount (GUI).

6.11 Registro de logs del contenedor de aplicación (ECS)

Acción realizada

Se configuró el driver de logs awslogs en la definición de tarea de ECS (task_definition.json), enviando los logs del contenedor aws-goat-m2 al log group /ecs/aws-goat-m2 de CloudWatch, con prefijo de stream aws-goat-m2, definido en el recurso aws_cloudwatch_log_group.ecs_logs con retención de 14 días.

```
GNU nano 7.2 task_definition.json *
    "value": "RDS_ENDPOINT_VALUE"
  },
  "linuxParameters": {
    "capabilities": {
      "add": [
        "SYS_PTRACE"
      ]
    },
    "initProcessEnabled": true
  },
  "cpu": 0,
  "image": "public.ecr.aws/p3q0v3y2/aws-goat-m2:latest",
  "name": "aws-goat-m2",
  "logConfiguration": {
    "logDriver": "awslogs",
    "options": {
      "awslogs-group": "/ecs/aws-goat-m2",
      "awslogs-region": "us-east-1",
      "awslogs-stream-prefix": "aws-goat-m2"
    }
  }
}
```

Figura 15. Extracto del archivo `task_definition.json` mostrando la sección `logConfiguration` con el driver `awslogs`, el log group `/ecs/aws-goat-m2` y el prefijo de stream configurados.

Se habilita así la trazabilidad de la actividad interna del contenedor, necesaria para el diagnóstico de incidentes.

6.12 Notificación centralizada mediante Amazon SNS

Acción realizada

Se creó el tema de SNS `aws-goat-m2-security-alerts` con una suscripción de tipo email hacia la dirección `cleidys.guzman@utp.ac.pa`, utilizado como destino común de las alarmas de cambios de IAM y de disponibilidad del servicio.

La suscripción fue verificada tanto por consola como por AWS CLI:

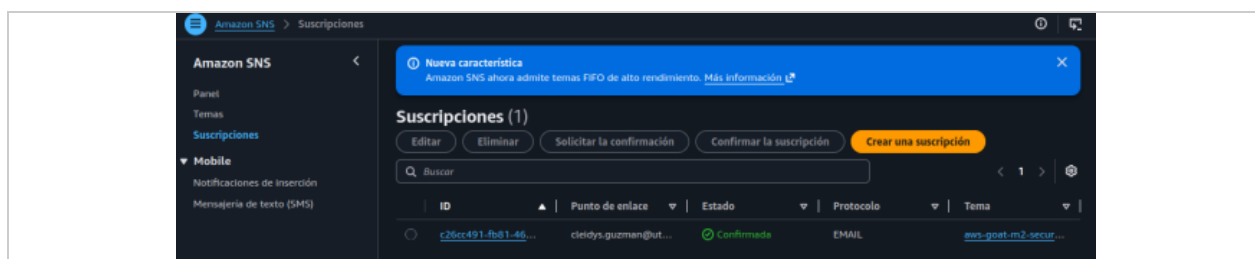


Figura 16. Consola de Amazon SNS — Suscripciones: una suscripción de tipo email, estado "Confirmada", asociada al tema `aws-goat-m2-security-alerts` (GUI).

```
ubuntu@ip-172-31-23-99:~$ aws sns list-subscriptions-by-topic --topic-arn arn:aws:sns:us-east-1:501578625392:aws-goat-m2-security-alerts
{
  "Subscriptions": [
    {
      "SubscriptionArn": "arn:aws:sns:us-east-1:501578625392:aws-goat-m2-security-alerts:c26cc491-fb81-4622-9269-bfb1f37db7a2",
      "Owner": "501578625392",
      "Protocol": "email",
      "Endpoint": "cleidys.guzman@utp.ac.pa",
      "TopicArn": "arn:aws:sns:us-east-1:501578625392:aws-goat-m2-security-alerts"
    }
  ]
}
```

Figura 17. Verificación por AWS CLI (`aws sns list-subscriptions-by-topic`) de la suscripción confirmada al tema `aws-goat-m2-security-alerts`, con el endpoint `cleidys.guzman@utp.ac.pa`.

6.13 Tagging y nomenclatura estándar

Problema identificado

Los recursos de la infraestructura original no contaban con un esquema de etiquetado consistente que permitiera identificar propietario, proyecto o ambiente.

Antes

```
provider "aws" {
  region = "us-east-1"
}
```

Después

```
provider "aws" {
  region = "us-east-1"
  default_tags {
    tags = {
      Project      = "AWSGoat-Modulo2"
      Environment  = "Lab"
      Owner        = "Cleidy-Guzman"
      ManagedBy    = "Terraform"
    }
  }
}
```

El bloque `default_tags` aplica automáticamente las cuatro etiquetas a todo recurso nuevo creado por Terraform. Esto exigió elevar la versión del proveedor de AWS de `~>3.27` a `~>3.75`, ya que `default_tags` requiere una versión igual o superior a 3.38. La verificación mediante AWS Resource Groups Tag Editor mostró 27 recursos correctamente etiquetados con Owner: Cleidy-Guzman.

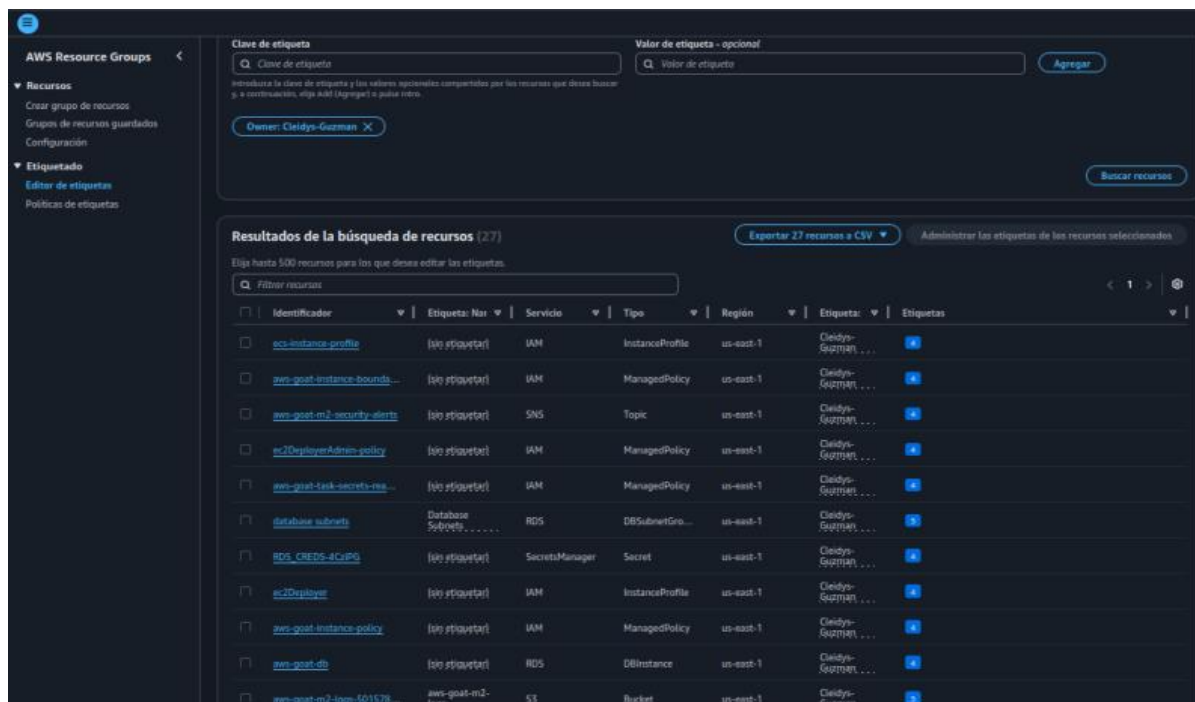


Figura 18. Consola de AWS Resource Groups (Tag Editor): 27 recursos de distintos tipos (IAM, SNS, RDS, S3, Secrets Manager, etc.) etiquetados con Owner: Cleidys-Guzman (GUI).

A nivel de AWS CLI se confirmó la presencia de las cuatro etiquetas tanto sobre el bucket de logs como sobre el log group de ECS:

```

ubuntu@ip-172-31-23-99:~$ aws s3api get-bucket-tagging --bucket aws-goat-m2-logs-501578625392
{
  "TagSet": [
    {
      "Key": "Project",
      "Value": "AWSGoat-Modulo2"
    },
    {
      "Key": "Environment",
      "Value": "Lab"
    },
    {
      "Key": "Owner",
      "Value": "Cleidy-Guzman"
    },
    {
      "Key": "ManagedBy",
      "Value": "Terraform"
    },
    {
      "Key": "Name",
      "Value": "aws-goat-m2-logs"
    }
  ]
}
ubuntu@ip-172-31-23-99:~$ aws logs list-tags-log-group --log-group-name /ecs/aws-goat-m2
{
  "tags": {
    "Project": "AWSGoat-Modulo2",
    "Owner": "Cleidy-Guzman",
    "ManagedBy": "Terraform",
    "Environment": "Lab"
  }
}

```

Figura 19. Verificación por AWS CLI de las etiquetas (Project, Environment, Owner, ManagedBy) sobre el bucket aws-goat-m2-logs-501578625392 (aws s3api get-bucket-tagging) y sobre el log group /ecs/aws-goat-m2 (aws logs list-tags-log-group).

Hallazgos Módulo 1

- Las cuatro vulnerabilidades de código asignadas (XSS reflejado, SQL Injection, IDOR y Sensitive Data Exposure) fueron remediadas mediante cambios acotados y de bajo riesgo: escapado automático de framework, consultas parametrizadas y validación de sesión mediante JWT, sin alterar la funcionalidad legítima de la aplicación.
- El pipeline de Terraform del laboratorio, en su configuración original, no garantizaba que un cambio de código fuente se reflejara en el entorno desplegado: la ausencia de source_code_hash en la función Lambda de datos y de etag en los objetos S3 del frontend hacía que Terraform reportara "No changes" pese a modificaciones reales en el código.
- Los provisioners local-exec (null_resource) utilizados para inyectar valores dinámicos (URL de API Gateway, nombre de bucket) en el build del frontend solo se ejecutan en la creación inicial del recurso, por carecer de un bloque triggers; cualquier recompilación posterior del frontend requiere repetir ese reemplazo manualmente.

- La verificación de cada control se realizó repitiendo el ataque original documentado, confirmando el cambio de comportamiento (de ejecución/exposición exitosa a bloqueo con 401 o neutralización del payload) directamente sobre el entorno desplegado en AWS, no solo sobre el código fuente.

Hallazgos Módulo 2

- La infraestructura original del Módulo 2 presentaba múltiples casos de exceso de privilegios en IAM, incluyendo una política administrada IAMFullAccess y una política comodín (Action "*", Resource "*") adjuntas a roles operativos, lo que representaba la debilidad de mayor severidad detectada.
- No existía ningún mecanismo de auditoría (CloudTrail) ni de monitoreo (CloudWatch) antes de la intervención del Blue Team, lo que habría impedido detectar actividad no autorizada en tiempo razonable.
- El bucket de estado de Terraform, que contiene información sensible de la infraestructura, no tenía protecciones básicas de S3 (bloqueo de acceso público, cifrado, versionado, logging).
- Las pruebas dirigidas de generación de alarmas (creación de usuario IAM de prueba y detención del servicio ECS) confirmaron que la cadena completa de detección y notificación — CloudTrail, CloudWatch Logs, metric filter, alarma y SNS — opera correctamente de extremo a extremo, con evidencia por CLI, por consola y por correo electrónico.
- El esquema de etiquetado (default_tags) se aplicó de forma consistente sobre 27 recursos de la cuenta, verificado mediante AWS Resource Groups Tag Editor y AWS CLI.
- El registro de acceso del ALB quedó confirmado no solo a nivel de configuración (CLI), sino de forma funcional, mediante la observación de archivos de log reales generados en el bucket de logs (consola).
- Dos de los controles aplicados a nivel de IAM (rol ec2-deployer y rol de tarea ECS sobre Secrets Manager) no cuentan con una prueba funcional documentada más allá del cambio en el código Terraform y su presencia en el inventario de recursos etiquetados; no se afirma su verificación operativa por falta de evidencia adicional.

Recomendaciones Módulo 1

- Declarar siempre source_code_hash (o equivalente) en todo recurso aws_lambda_function cuyo código se gestione mediante data "archive_file", para que Terraform detecte y propague automáticamente cualquier cambio de código fuente.
- Declarar etag = filemd5(...) (o utilizar source_hash) en todo aws_s3_object cuyo contenido pueda cambiar entre aplicaciones sucesivas de Terraform, especialmente en despliegues de artefactos compilados (builds de frontend).
- Evitar depender de provisioners local-exec sin bloque triggers para operaciones que deban repetirse ante cambios de contenido; considerar en su lugar variables de entorno inyectadas en tiempo de build o un paso de compilación explícito dentro del pipeline de CI/CD.
- Evitar el uso de dangerouslySetInnerHTML (o equivalentes en otros frameworks) para renderizar cualquier valor derivado de input de usuario; extender la revisión aplicada en este ejercicio a los dos usos adicionales detectados en blogpost.js y HomePage.js.

- Adoptar consultas parametrizadas como estándar para toda interacción con bases de datos (SQL o PartiQL), prohibiendo la concatenación de input de usuario en sentencias de consulta durante la revisión de código.
- Derivar siempre la identidad del sujeto de una operación sensible (cambio de contraseña, acceso a datos propios) del token de sesión validado en el servidor, nunca de un campo del cuerpo de la petición controlable por el cliente.
- Exigir autenticación y autorización explícitas en todo endpoint que exponga datos de usuarios, incluyendo endpoints de utilidad interna (como /dump), que no deberían existir en un entorno productivo sin controles adicionales.
- Incorporar una prueba de re-testeo (repetir el ataque original tras cada remediación) como paso obligatorio del proceso de Blue Team, dado que en este ejercicio se detectó que un parche de código correcto no garantiza por sí solo la remediación si el pipeline de despliegue no lo propaga.

Recomendaciones Módulo 2

Con base en las vulnerabilidades identificadas y los controles aplicados, se plantean las siguientes recomendaciones para un entorno productivo de AWS:

- Evitar por completo el uso de políticas IAM con Action "*" o Resource "*"; definir siempre el conjunto mínimo de acciones y recursos necesarios para cada rol.
- Revisar periódicamente las políticas de permissions boundary y los permisos de iam:PassRole, restringiéndolos siempre a roles y condiciones específicas, como se aplicó en este ejercicio.
- Extender el uso de políticas de solo lectura o de alcance limitado (por ARN de recurso) a todo servicio que interactúe con Secrets Manager, en lugar de políticas de lectura/escritura genéricas.
- Habilitar cifrado en reposo, bloqueo de acceso público, versionado y logging en todo bucket S3 desde el momento de su creación, idealmente mediante políticas de organización (AWS Organizations / Service Control Policies) que lo exijan.
- Configurar CloudTrail como trail multi-región desde el inicio del proyecto, en lugar de limitarlo a una sola región, para cubrir actividad en toda la cuenta.
- Ampliar el conjunto de métricas y alarmas de seguridad más allá de los eventos de IAM y disponibilidad, incorporando por ejemplo cambios en Security Groups o en políticas de buckets S3.
- Evaluar la incorporación de servicios adicionales de seguridad de AWS no utilizados en este laboratorio, como AWS Config para evaluación continua de cumplimiento o Amazon GuardDuty para detección de amenazas.
- Establecer un proceso de rotación periódica de credenciales almacenadas en Secrets Manager, en particular las credenciales de la base de datos RDS.
- Mantener el esquema de etiquetado (tagging) como práctica obligatoria desde el diseño de la infraestructura, no como corrección posterior, para facilitar la gobernanza y la atribución de costos y responsabilidades.

Conclusiones Módulo 1

El proceso de remediación aplicado sobre los cuatro primeros retos del Módulo 1 de AWSGoat permitió corregir vulnerabilidades representativas de las categorías A01 (Broken Access Control) y A03 (Injection) del OWASP Top 10:2021, mediante cambios de código acotados: escapado de salida en el frontend, parametrización de consultas y validación de sesión basada en JWT en el backend.

Más allá de las vulnerabilidades originalmente asignadas, el ejercicio evidenció un riesgo adicional propio del proceso de remediación: un pipeline de Infraestructura como Código que no vincula correctamente el contenido de los artefactos desplegados (código Lambda, build de frontend) al estado gestionado por Terraform puede dar una falsa sensación de que un parche fue aplicado, cuando en realidad el entorno productivo continúa ejecutando la versión vulnerable. Este hallazgo refuerza la importancia de verificar toda remediación directamente contra el entorno desplegado —y no únicamente contra el código fuente— antes de darla por concluida.

En conjunto, los cuatro controles aplicados y verificados, junto con la corrección del pipeline de despliegue, dejan el Módulo 1 en un estado donde las vulnerabilidades explotadas por el Red Team durante la fase de ataque ya no son reproducibles sobre el entorno actualmente desplegado.

Conclusiones Módulo 2

El proceso de remediación aplicado sobre el Módulo 2 de AWSGoat permitió transformar una infraestructura con permisos administrativos extendidos, ausencia total de auditoría y almacenamiento sin protección, en un entorno con privilegios acotados por rol, trazabilidad de eventos mediante CloudTrail y CloudWatch, y controles de protección de datos en reposo sobre sus buckets S3.

La postura de seguridad de la infraestructura mejoró de forma verificable: la mayoría de las correcciones aplicadas sobre el código Terraform fueron validadas mediante consola de AWS o AWS CLI, y los dos mecanismos de alertamiento implementados (cambios de IAM y disponibilidad del servicio) fueron probados de forma activa, confirmando notificaciones reales por correo electrónico ante eventos simulados.

Este ejercicio evidencia la importancia de aplicar controles de seguridad desde el diseño de la infraestructura (security by design) y no como una capa añadida posteriormente, así como la necesidad de un monitoreo continuo que permita detectar y responder oportunamente ante cambios no autorizados o incidentes de disponibilidad en un entorno de nube.